

KOCAELİ ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ
PROGRAMLAMA LABORATUVARI - II
III. Proje

TESLİM TARİHİ: 03.06.2026

NoSQL'den SQL'e Dönüşüm Sistemi

Bu projede öğrencilerden, yapısı önceden bilinmeyen karmaşık JSON (NoSQL) verilerini dinamik olarak analiz edip ilişkisel SQL veritabanına dönüştüren jenerik bir uygulama geliştirmeleri beklenmektedir. Bu uygulama, kendisine verilen herhangi bir iç içe geçmiş (nested) JSON belgesini okuyacak, hiyerarşik yapısını algoritmik olarak çözümleyecek ve otomatik olarak uygun SQL tabloları ile aralarındaki ilişkileri (Primary/Foreign Key) kurarak veriyi ilişkisel yapıya aktaracaktır.

Projenin amacı sadece sabit bir veriyi bir yerden başka bir yere kopyalamak değildir. Amaç, tek bir proje içinde öğrencilerin NoSQL'in esnek, şemasız (schema-less) yapısı ile SQL'in katı, normalize edilmiş tablo yapısı arasındaki temel farkları uygulamalı olarak görmeleri ve bu iki farklı paradigma arasında otonom çalışan bir dönüştürücü (parser/mapper) geliştirmelerini sağlamaktır.

Projede, farklı derinliklerde iç içe objeler ve listeler (arrays) içeren, sistemin sınırlarını zorlayacak çeşitli JSON test dosyaları kullanılacaktır. Öğrencilerin geliştireceği algoritma; JSON içindeki listeleri (array) tespit edip onlar için otomatik olarak yeni alt tablolar (1:N ilişkisi) oluşturmalı ve iç içe objeleri tek bir tabloda sütunlar halinde düzleştirmelidir (flattening). Bu nedenle öğrencilerin kodu, tek bir senaryoya veya sabit bir JSON yapısına bağımlı (hardcoded) olmamalı; farklı veri yapılarına uyum sağlayabilen genelgeçer ve esnek bir mimaride tasarlanmalıdır.

Amaç

Bu projenin temel amacı, öğrencilerin veri modelleme kavramlarını derinlemesine anlamasını sağlamak ve esnek yapı, şemasız (schema-less) NoSQL (JSON) veri modelleri ile katı kurallı, normalize edilmiş ilişkisel SQL veri modelleri arasında algoritmik bir köprü kurmalarını sağlamaktır.

Öğrencilerden beklenen, sadece belirli bir veriyi kopyalayan statik bir program yazmaları değil; yapısı, içerdiği alanlar ve derinliği önceden bilinmeyen karmaşık JSON dosyalarını okuyarak, ilişkisel veritabanı kurallarına uygun tabloları dinamik olarak üreten jenerik bir dönüştürücü motor geliştirmeleridir.

Projenin Genel Yapısı

Bu projede geliştirilecek sistem, veritabanı tablolarının önceden elle kodlanmadığı (hardcoded olmadığı), tamamen gelen veriye göre şekil alan dinamik bir dönüştürücü motor (converter engine) olarak çalışacaktır. Sistemin genel yapısı; veri girişinin alınması, hiyerarşinin çözülmesi, SQL şemasının çalışma zamanında (runtime) oluşturulması ve verinin aktarılması olmak üzere ardışık 4 temel aşamadan oluşur.

Öğrencilerin geliştireceği uygulamanın mimarisi ve arka plandaki çalışma akışı şu şekilde

olmalıdır:

1. Veri Okuma ve Hiyerarşi Analizi (Parsing)

- Sisteme, yapısı ve derinliği önceden bilinmeyen karmaşık bir JSON dosyası girdi olarak verilir.
- Uygulama, bu JSON dosyasını belleğe okur ve içindeki veri tiplerini (metin, sayı, obje, dizi) anahtar-değer (key-value) ilişkilerine göre tarar.

2. Düzleştirme (Flattening) ve Ana Tablo Tasarımı

- Sistem, basit veri tiplerini (String, Integer vb.) doğrudan oluşturulacak ana tablonun sütunları olarak belirler.
- Eğer okunan JSON içerisinde iç içe geçmiş bir alt obje (nested object) bulunursa, uygulama bu hiyerarşiyi SQL'e uygun hale getirmek için "**düzleştirme**" işlemi yapar. Örneğin; "adres": {"sehir": "Kocaeli", "ilce": "İzmit"} şeklindeki bir yapı, veritabanında adres_sehir ve adres_ilce adında iki ayrı sütuna dönüştürülür.

3. Otomatik Normalizasyon ve İlişki Kurma (Handling Arrays)

- Sistemin en kritik aşamasıdır. Uygulama veriyi tararken bir liste/dizi (array) yapısı ile karşılaşır, ilişkisel veritabanı kuralları gereği (1NF) bu listeyi aynı tabloya sıkıştırmaya çalışmaz.
- Algoritma diziyi fark ettiği anda işlemi durdurur ve bu dizideki elemanları tutmak için otomatik olarak **yeni bir alt tablo (child table)** oluşturur.
- Daha sonra, yeni oluşturulan bu alt tablo ile ana tablo arasında veri bütünlüğünü sağlamak için bir Yabancı Anahtar (Foreign Key) bağlantısı kurarak **Bire-Çok (1:N) ilişkisini** tanımlar.

4. SQL Sorgularının Üretilmesi ve Veri Aktarımı (Execution)

- Uygulama, JSON'un tüm yapısını analiz edip sanal bir tablo taslağı çıkardıktan sonra, bu taslağı gerçek SQL komutlarına dönüştürür.
- İlk olarak, analiz edilen yapıya uygun olarak tabloları ve birbirleri arasındaki Foreign Key ilişkilerini tanımlayan CREATE TABLE sorgularını dinamik olarak üretip çalıştırır.
- Tablolar oluştuktan sonra, JSON'daki ham verileri algoritmik olarak parçalar ve ilgili alt/üst tablolara INSERT INTO sorguları ile hatasız bir şekilde kaydeder.

Öğrencilerden Beklenen Görevler ve Sistem Modülleri

Bu projeyi başarıyla tamamlamak için öğrencilerin aşağıdaki modülleri ve fonksiyonları eksiksiz bir şekilde kodlamaları gerekmektedir. Proje bir masaüstü uygulaması olarak tasarlanacak ve arka planda dinamik bir dönüşüm algoritması çalıştırılacaktır.

1. Veri Okuma ve Görselleştirme Modülü

- **Dosya Seçimi:** Kullanıcı arayüzü (GUI) üzerinden herhangi bir JSON dosyasının (önceden yapısı bilinmeyen) seçilip sisteme yüklenmesi sağlanmalıdır.
- **Ağaç Görüntüleme (Tree View):** Yüklenen JSON verisinin hiyerarşik yapısı (objeler ve alt diziler), arayüzde okunabilir bir ağaç yapısında veya formatlı bir metin (pretty-print) olarak kullanıcıya gösterilmelidir.

2. Algoritmik Çözümleme (Parsing) Modülü

- **Veri Tipi Tespiti:** JSON içerisindeki her bir anahtarın (key) taşıdığı değerın metin, sayı, boolean, obje veya dizi olup olmadığını program otomatik olarak tespit etmelidir.

- Düzleştirme (Flattening) İşlemi: İç içe geçmiş obje ({ . . . }) yapılarındaki alanlar, ebeveyn ve çocuk anahtarları birleştirilerek (örn: ebeveyn_cocuk formatında) tek bir satır düzlemine indirilmelidir.

3. Dinamik Veritabanı Motoru Modülü (Projenin Kalbi)

- Otomatik Tablo Tasarımı: Program, JSON yapısına bakarak SQL tablolarının isimlerine, sütun adlarına ve SQL veri tiplerine (VARCHAR, INTEGER vb.) otonom olarak karar vermelidir. Öğrenciler tablo isimlerini ve sütun yapılarını koda manuel (hardcoded) olarak kesinlikle yazmamalıdır.
- Liste (Array) Yönetimi ve Normalizasyon: Program bir dizi ([. . .]) ile karşılaştığında, ana tabloyu bölmeli, bu dizi içindeki elemanları saklamak için yeni bir alt tablo oluşturmalı ve ana tablonun ID'sini alt tabloya **Foreign Key** olarak atamalıdır.

1NF (Birinci Normal Form) Çoklu Değerlerin Ayrıştırılması	Algoritma, JSON hiyerarşisi içerisinde bir dizi (array) yapısı ile karşılaştığında, bu verileri ana tabloya gömmek yerine otomatik olarak yeni bir alt tablo (Child Table) oluşturur. Her bir dizi elemanı bu alt tabloda ayrı birer satır olarak tutulur, böylece atomik veri kuralı sağlanmış olur.
2NF (İkinci Normal Form) Tam Bağımlılığın Sağlanması	Dinamik olarak oluşturulan her tablo için otomatik bir Primary Key (Birincil Anahtar) atanır. Alt tablolar ile ana tablo arasındaki ilişki, sistem tarafından otomatik üretilen Foreign Key (Yabancı Anahtar) bağlantılarıyla kurulur; böylece her verinin sadece ilgili olduğu anahtara tam bağımlı olması sağlanır.
3NF (Üçüncü Normal Form) Geçişli Bağımlılıkların Giderilmesi	Sistem, 5-6 katman derinliğe sahip verileri analiz ederken geçişli bağımlılıkları (transitive dependency) tespit eder. Örneğin; "Şehir -> İlçe -> Mahalle" gibi iç içe geçmiş nesne grupları, hiyerarşik Foreign Key yapılarıyla birbirine bağlı ayrı tablolara bölünerek veri tekrarı önlenir.

4. SQL Yürütme ve Aktarım Modülü

- **Sorgu Üretimi:** Oluşturulan sanal tablo taslakları, gerçek CREATE TABLE sorgularına dönüştürülüp seçilen veritabanında (örn. SQLite) çalıştırılmalıdır.
- **Veri Yazma:** Ayrıştırılan tüm ham veriler, doğru tablolar ve sütunlar eşleştirilerek INSERT INTO sorguları ile hatasız bir şekilde veritabanına aktarılmalıdır.

5. Raporlama ve Arayüz Modülü

- **SQL Tablo Gösterimi:** Dönüşüm işlemi tamamlandıktan sonra, veritabanında dinamik olarak oluşturulan tablolar ve içerikleri arayüzde bir tablo (DataGrid/Table) yapısında listelenmelidir.
- **Sıfırlama Butonu:** Yeni bir JSON dosyası denemek ve sistemi sıfırlamak için, arayüzde veritabanındaki tüm tabloları silen (DROP TABLE) bir temizleme butonu bulunmalıdır.

Proje Teslimi

- Rapor **IEEE** formatında (önceki yıllarda verilen formatta) 4 sayfa, akış diyagramı veya yalancı kod içeren, özet, giriş, yöntem, örnek görseller, sonuç ve kaynakça bölümünden oluşmalıdır. Raporda oluşturulan veritabanına ait **class diyagramları** ve **use case diyagramları** bulunmalıdır.
- Dersin takibi projenin teslimi dahil **edestek2.kocaeli.edu.tr** sistemi üzerinden yapılacaktır. edestek2.kocaeli.edu.tr sitesinde belirtilen tarihten sonra getirilen projeler kabul edilmeyecektir.
- Proje ile ilgili sorular edestek.kocaeli.edu.tr sitesindeki forum üzerinden **Arş. Gör. Neslihan Bengisu Çolak** veya **Arş. Gör. Yılmaz Dikilitaş**'a sorulabilir.
- Demo sırasında kullandığınız algoritma, geliştirdiğiniz kodun çeşitli kısımlarının ne amaçla yazıldığı ve geliştirme ortamı hakkında sorular sorulabilir.
- Yazdığınız herhangi bir satır kodu açıklamanız istenebilir.
- Proje grupları **her öğretim kendi içinde** olmak üzere **en fazla iki kişiden** oluşturulmalıdır.